Intermediate (real branching) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

`add_conditional_edges` takes 3 arguments: the node the decision happens after, a function that reads state and returns a string, and a dict mapping each possible returned string to the real node it should lead to. The routing function itself is **not** a node — it doesn't get registered with `add_node`, and it doesn't return a state update. Its only job is picking a destination.

The exact pieces:

- `class GraphState(TypedDict): flag: bool; message: str` — a boolean field the routing function will branch on.
- `def route(state: GraphState) -> str:` with `if state["flag"]: return "path_a"`, then `return "path_b"` — reads state, returns a plain string naming the chosen path.
- `builder.add_conditional_edges("decide", route, {"path_a": "node_a", "path_b": "node_b"})` — wires it up: after node `"decide"` runs, call `route`, then go to whichever real node the returned string maps to.
- Both `node_a` and `node_b` still need their own `add_edge(..., END)` (or a further conditional edge) — a conditional edge only decides *where to go next*, it doesn't make either branch automatically finish the graph.
- Invoke twice, once with `{"flag": True, ...}` and once with `{"flag": False, ...}`, and confirm each one actually reaches the node you expect.

Think about what happens the moment `route` has a bug — say it returns `"path_c"` by mistake, a string that isn't a key in your mapping at all (this is exactly what the Edge cases exercise digs into). Plain Python gives you no protection here: `route`'s return type is just `str`, so nothing stops it from returning *any* string, including one your mapping was never built to handle.

The real design question isn't just "how do I wire a routing function" — it's "how do I make it hard for the routing function to return a value that isn't actually one of my real options?"

The extra piece that helps:

- `from typing import Literal` and `def route(state: GraphState) -> Literal["path_a", "path_b"]:` — this doesn't stop a bug at runtime by itself, but it does mean a type checker (or your editor) flags it immediately if you write `return "path_c"` or misspell `"path_a"` somewhere — catching the mistake while you're writing the code, not after you've run it and gotten a confusing failure.

This is worth doing any time a routing function's return value has to exactly match a fixed set of dict keys somewhere else in the file — the two are only connected by string-matching, and `Literal` is the cheapest way to make that connection visible to your tools instead of silent.

Hint 2 — The plan, and almost the whole thing

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

define:
    class GraphState(TypedDict):
        flag: bool
        message: str

define:
    def node_decide(state: GraphState) -> dict:
        return {}

    def node_a(state: GraphState) -> dict:
        return {"message": "went down path A"}

    def node_b(state: GraphState) -> dict:
        return {"message": "went down path B"}

    def route(state: GraphState) -> str:
        if state["flag"]:
            return "path_a"
        return "path_b"

build:
    builder = StateGraph(GraphState)
    builder.add_node("node_decide", node_decide)
    builder.add_node("node_a", node_a)
    builder.add_node("node_b", node_b)
    builder.add_edge(START, "node_decide")
    builder.add_conditional_edges(
        "node_decide", route, {"path_a": "node_a", "path_b": "node_b"},
```

```
    )
    builder.add_edge("node_a", END)
    builder.add_edge("node_b", END)

run:
    graph = builder.compile()
    print(graph.invoke({"flag": True, "message": ""}))
    print(graph.invoke({"flag": False, "message": ""}))
```

Trace it by hand before running: with `flag=True`, which node should run after `node_decide`? Then run it and compare against the Solution.

Once that's working, narrow `route`'s return type so a typo can't silently break it:

```
same graph as Intermediate, but:

from typing import Literal

def route(state: GraphState) -> Literal["path_a", "path_b"]:
    if state["flag"]:
        return "path_a"
    return "path_b"
```

Run a type checker (or just read carefully) over a *deliberately broken* version where the mapping only has `"path_a"` and `route` can still return `"path_b"` — confirm your editor or `mypy` flags the mismatch before you ever run the code. Then fix the mapping and confirm both branches still work exactly as before.

Full solution: Show me the solution